

■ Lab 8 — Expressions Conditionnelles

■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Utilisé l'expression conditionnelle `condition ? true_val : false_val`
- Adapté l'infrastructure selon l'**environnement** (dev / prod)
- Combiné conditionnels avec des **variables** et des **locals**

■ Étape 1 — Créer les fichiers

```
cd labs/lab8-conditionals
```

``backend.tf``

```
terraform {
  backend "s3" {
    bucket = "tf-training-<votre-prenom>-982908300187"
    key    = "terraform.tfstate"
    region = "eu-west-1"
    use_lockfile = true
    encrypt      = true
  }
}
```

■■ Remplacez `` par votre prénom. Ex : `tf-training-alice-982908300187`

``versions.tf``

```
terraform {
  required_version = ">= 1.15.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

``provider.tf``

```
provider "aws" {  
  region = var.region  
}
```

`variables.tf`

```
variable "username" {  
  description = "Votre prenom"  
  type        = string  
}  
  
variable "region" {  
  description = "Region AWS"  
  type        = string  
  default     = "eu-west-1"  
}  
  
variable "environment" {  
  description = "Environnement de déploiement"  
  type        = string  
  default     = "dev"  
  
  validation {  
    condition     = contains(["dev", "prod"], var.environment)  
    error_message = "L'environnement doit être dev ou prod."  
  }  
}
```

`locals.tf`

```
locals {  
  prefix = "lab8-${var.username}-${var.environment}"  
  
  # Conditionnel : type d'instance selon l'environnement  
  instance_type = var.environment == "prod" ? "t2.small" : "t2.micro"  
  
  # Conditionnel : nombre d'instances selon l'environnement  
  instance_count = var.environment == "prod" ? 2 : 1  
  
  # Conditionnel : monitoring activé uniquement en prod  
  monitoring_enabled = var.environment == "prod" ? true : false  
  
  common_tags = {  
    Lab           = "lab8"  
    Username      = var.username  
    Environment   = var.environment  
    ManagedBy     = "Terraform"  
  }  
}
```

■ La syntaxe est : `condition ? valeur_si_vrai : valeur_si_faux`

La condition peut être n'importe quelle expression qui retourne un booléen.

`main.tf`

```
resource "aws_security_group" "lab8_sg" {
  name          = "${local.prefix}-sg"
  description    = "Security group Lab 8 - ${var.username} - ${var.environment}"

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-sg"
  })
}

resource "aws_instance" "lab8_instance" {
  count = local.instance_count

  ami          = "ami-0694d931cee176e7d"
  instance_type = local.instance_type
  monitoring    = local.monitoring_enabled
  vpc_security_group_ids = [aws_security_group.lab8_sg.id]

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-instance-${count.index + 1}"
  })
}
```

`outputs.tf`

```
output "instance_ids" {
  description = "IDs des instances"
  value       = aws_instance.lab8_instance[*].id
}

output "instance_type_used" {
  description = "Type d'instance utilisé"
  value       = local.instance_type
}

output "instance_count_used" {
  description = "Nombre d'instances déployées"
  value       = local.instance_count
}

output "monitoring_enabled" {
  description = "Monitoring activé"
  value       = local.monitoring_enabled
}
```

■ Étape 2 — Déployer en dev

```
terraform init
terraform apply -var="username=<votre-prenom>" -var="environment=dev"
```

Vérifiez les outputs :

```
terraform output instance_type_used
terraform output instance_count_used
terraform output monitoring_enabled
```

Résultats attendus en **dev** :

```
instance_type_used  = "t2.micro"
instance_count_used = 1
monitoring_enabled  = false
```

■ Étape 3 — Déployer en prod

```
terraform apply -var="username=<votre-prenom>" -var="environment=prod"
```

```
terraform output instance_type_used
terraform output instance_count_used
terraform output monitoring_enabled
```

Résultats attendus en **prod** :

```
instance_type_used  = "t2.small"
instance_count_used = 2
monitoring_enabled  = true
```

■ Terraform adapte automatiquement le type d'instance, le nombre et le monitoring selon l'environnement — sans modifier une seule ligne de `main.tf`.

■ Étape 4 — Tester avec terraform console

```
terraform console -var="username=<votre-prenom>" -var="environment=prod"
```

```
> local.instance_type
"t2.small"

> local.instance_count
2

> local.monitoring_enabled
true

> var.environment == "prod" ? "PRODUCTION" : "DÉVELOPPEMENT"
"PRODUCTION"
```

Tapez `exit` pour quitter.

■ Étape 5 — Nettoyage

```
terraform destroy -var="username=<votre-prenom>" -var="environment=prod"
```

■ Validation du Lab

#	Critère	Validé
1	`instance_type_used` retourne `t2.micro` en dev	■
2	`instance_type_used` retourne `t2.small` en prod	■
3	`instance_count_used` retourne `1` en dev et `2` en prod	■
4	`monitoring_enabled` retourne `false` en dev et `true` en prod	■
5	`terraform console` évalue les conditionnels correctement	■
6	`terraform destroy` supprime toutes les ressources	■

■ ****Fin du Lab 8 — Votre infrastructure s'adapte selon l'environnement !****

*Le Lab 9 introduit les **dynamic blocks** pour itérer sur des attributs de type block.*